

# ANÁLISIS MULTICAPA DE OPERADORES DE POSTGRESQL Y ALMACENAMIENTO CSI EN KUBERNETES: UN MARCO DE ANÁLISIS Y UN ESTUDIO EMPÍRICO DE CLOUDNATIVEPG BAJO FALLOS INYECTADOS

## Multilayer Analysis of PostgreSQL Operators and CSI Storage in Kubernetes: An Analytical Framework and an Empirical Study of CloudNativePG under Injected Faults

Angel A. Parejo R.<sup>1</sup>

<sup>1</sup>Universidad de Carabobo, Valencia, Venezuela

angelparejo@gmail.com ORCID: 0009-0001-9737-7116

\*Autor de correspondencia: Angel A. Parejo R.; angelparejo@gmail.com

**Resumen.** La adopción de Kubernetes para cargas con estado ha consolidado el uso de los operadores de PostgreSQL y de *Container Storage Interface* (CSI), cuya interacción ante fallos rara vez se evalúa conjuntamente. Este artículo presenta un marco descriptivo —taxonomía, vocabulario común y el modelo  $S = (O, K, M, D)$  con invariantes de consistencia, disponibilidad y durabilidad— y un estudio empírico de CloudNativePG mediante inyección de fallos en un clúster productivo bajo eliminación del primario (F1), indisponibilidad sostenida (F2) y partición de red (F3). El hallazgo central es que la variable que gobierna el *failover* no es el fallo, sino su visibilidad ante Kubernetes: eliminar el pod primario promueve una réplica (RTO mediano 7,91 s), mientras que el fallo sostenido no promueve (36,75 s) y la partición preserva la consistencia. Ninguna transacción confirmada se perdió (RPO nulo). Por la colocación intranodo, se sostienen el contraste F1/F2 y el mecanismo, no las magnitudes absolutas.

**Palabras clave:** CloudNativePG; conmutación por error; inyección de fallos; Kubernetes; PostgreSQL.

**Abstract.** The adoption of Kubernetes for stateful workloads has consolidated the use of PostgreSQL operators and of the *Container Storage Interface* (CSI), whose interaction under failure is rarely evaluated jointly. This paper presents a descriptive framework —a taxonomy, a common vocabulary, and the model  $S = (O, K, M, D)$  with consistency, availability, and durability invariants— and an empirical study of CloudNativePG through fault injection on a production cluster under primary deletion (F1), sustained unavailability (F2), and network partition (F3). The central finding is that the variable governing failover is not the fault but its visibility to Kubernetes: deleting the primary pod promotes a replica (median RTO 7.91 s), whereas the sustained fault does not promote (36.75 s) and the partition preserves consistency. No committed transaction was lost (null RPO). Owing to intra-node co-location, the F1/F2 contrast and the mechanism hold, not the absolute magnitudes.

**Keywords:** CloudNativePG; failover; fault injection; Kubernetes; PostgreSQL.

## 1. Introducción

Kubernetes se ha convertido en la plataforma de facto para la orquestación de cargas de trabajo en contenedores, incluyendo aplicaciones con estado como las bases de datos (Schwartzkopf et al., 2013; Burns et al., 2016). Este avance ha

favorecido el desarrollo de operadores de bases de datos, particularmente para PostgreSQL, como un mecanismo para automatizar la gestión del ciclo de vida, la replicación y los procesos de conmutación por error (*failover*) (Dobies & Wood, 2020; Kubernetes, 2026b).

En paralelo, la *Container Storage Interface* (CSI) estandariza la integración de *backends* de almacenamiento heterogéneos (Kubernetes, 2024). La literatura, sin embargo, tiende a analizar estos componentes de forma aislada —orquestación, consistencia distribuida o arquitectura de bases de datos (Gilbert & Lynch, 2002; Bailis & Ghodsi, 2013; Khan et al., 2023)—, pese a que la resiliencia y la recuperación ante fallos dependen de la interacción entre capas (Avizienis et al., 2004; Burckhardt, 2014).

El argumento central de este artículo es que la confiabilidad de las cargas con estado en Kubernetes es un problema inherentemente multicapa: aunque un operador implemente una lógica de control correcta, su recuperación está condicionada por la latencia, la durabilidad, la topología y el *failover* del *backend*, y una capa de almacenamiento bien configurada no basta sin alineación con los supuestos del operador y la semántica de replicación. En este contexto, el trabajo presenta:

- Una taxonomía de operadores de PostgreSQL y de almacenamiento basado en CSI en Kubernetes.
- Un marco descriptivo con vocabulario común —el modelo  $S = (O, K, M, D)$  e invariantes de durabilidad, disponibilidad y consistencia—, sin pretensión predictiva.
- El hallazgo empírico central: mediante inyección controlada de fallos sobre un clúster productivo, la variable que gobierna el *failover* en CloudNativePG no es el fallo sino su *visibilidad* ante Kubernetes; en particular, la indisponibilidad sostenida del primario (*pod-failure*) no promueve réplica, en contra del comportamiento documentado —una *refutación*, no una confirmación—.
- La propuesta de extender el marco con un predicado de visibilidad  $V(\text{fallo}, K)$  —pod eliminado, *NotReady*-recreado o *Ready*—.

El estudio empírico es una fase piloto (Fase 1) cuyo alcance está delimitado por las restriccio-

nes del clúster productivo (Sección 5).

## 2. Trabajos relacionados

La literatura se agrupa en tres líneas con un marco común. La *orquestación de clústeres* arranca en Borg y Omega, base de Kubernetes (Schwartzkopf et al., 2013; Burns et al., 2016), donde la CSI abstrae el almacenamiento persistente (Kubernetes, 2024) y el patrón *Operator* consolida la gestión declarativa (Kubernetes, 2026b). En *bases de datos distribuidas* destacan el teorema CAP y sus relecturas (Gilbert & Lynch, 2002; Bailis & Ghodsi, 2013), la consistencia eventual fuerte (Burckhardt, 2014) y su materialización en CockroachDB (Taft et al., 2020). Estas propiedades se nombran según la taxonomía canónica de la *dependability* y su cadena falta–error–fallo (Avizienis et al., 2004). La tercera línea, *recuperación y replicación*, aporta el WAL (Stonebraker & Kemnitz, 1991; Han et al., 2024) y el efecto de la replicación en el *failover* (Van Renesse & Schneider, 2004). El despliegue de sistemas *stateful* mediante operadores ha ganado atención (Dobies & Wood, 2020; Mega, 2023), aunque rara vez caracteriza el comportamiento ante fallos inyectados, brecha que este trabajo aborda.

Una cuarta vertiente, metodológica, verifica la resiliencia mediante inyección controlada de fallos: el *chaos engineering* (Basiri et al., 2016), la inyección dirigida por linaje (Alvaro & Tymon, 2018) y la caracterización empírica de particiones de red en producción (Alquaraan et al., 2018). En la verificación de consistencia y durabilidad de bases de datos, la metodología Jepsen y su verificador Elle (Kingsbury & Alvaro, 2021) son la referencia canónica; el *tx-verifier* de este trabajo comparte ese espíritu con un alcance más acotado —durabilidad transaccional a nivel de *commit*, contigüidad de una secuencia monótona como *proxy* del RPO— sobre un operador concreto. El propio CloudNativePG incorpora pruebas de caos en su integración continua (LitmusChaos elimina el primario repetidamente) para validar el *failo-*

ver (Drees, 2026), pero sin cuantificar su coste ni distinguir comportamientos; y Chen et al. (2026) evalúan la resiliencia del plano de orquestación, no de un operador con estado. Existen además comparaciones de industria entre operadores (Engelbert, 2024; Portworx, 2024), limitadas a listas de características. La brecha que este artículo cierra no es la ausencia de inyección de fallos en Kubernetes, sino la combinación específica de operador de PostgreSQL, almacenamiento CSI y un verificador a nivel de *commit* que cuantifica la durabilidad y la consistencia observables, con la interacción entre capas —poco abordada— en el centro.

### 3. Modelo del sistema y marco de evaluación

#### 3.1. Taxonomía de operadores y almacenamiento

Se clasifican los componentes según su diseño operativo. Entre los operadores de PostgreSQL se distinguen tres enfoques: CloudNativePG (CNPG), *cloud-native*, integra la lógica del operador con las primitivas y los bucles de reconciliación de Kubernetes; Zalando Postgres Operator depende de herramientas externas de alta disponibilidad (Patroni) como coordinación; y Crunchy Postgres for Kubernetes adopta una posición intermedia. El estudio empírico se limita a CNPG por una restricción del entorno productivo (Sección 5); el contraste con los demás se aborda analíticamente en la Tabla 1. Estas diferencias influyen en cómo se detectan fallos, se ejecuta el *failover* y se gestionan las transiciones de estado (Dobies & Wood, 2020; Kubernetes, 2026b).

La CSI respalda tres categorías de almacenamiento (Kubernetes, 2024): local (baja latencia, mayor sensibilidad a fallos de nodo), distribuido —Ceph, Longhorn— (tolerancia a fallos por replicación, a costa de latencia) y en red (desacoplamiento topológico, comportamiento dependiente de la red). Cada categoría anticipa, a partir de su comportamiento documentado, un modo de recuperación distinto ante el mismo fallo —expectativa que la Sección 6 somete a

prueba para CNPG—.

#### 3.2. Modelo del sistema

A partir de la taxonomía descrita, el sistema se modela como una tupla, según la Ec. (1):

$$S = (O, K, M, D) \quad (1)$$

Donde  $O$  es la lógica del operador,  $K$  el plano de control de Kubernetes,  $M$  la capa de almacenamiento gestionada mediante CSI —se usa  $M$ , no  $C$ , para no colisionar con la consistencia  $C(S)$  de la Sección 3.3— y  $D$  el motor de base de datos (Schwarzkopf et al., 2013; Burns et al., 2016; Kubernetes, 2024). Más que una formulación matemática, es un vocabulario descriptivo —andamiaje organizativo, no un modelo predictivo ya validado— que permite razonar sobre cómo las decisiones en una capa se propagan al comportamiento global bajo fallo.

#### 3.3. Dimensiones de evaluación

Con base en el modelo anterior, se definen cuatro dimensiones principales que permiten evaluar el comportamiento del sistema desde diferentes perspectivas.

*Resiliencia*: se modela según la Ec. (2),

$$R(S) = f(F, T_{\text{recuperación}}, A), \quad (2)$$

con  $F$  el tipo de fallo,  $T_{\text{recuperación}}$  el tiempo de restablecimiento y  $A$  la disponibilidad tras la recuperación; especializa la taxonomía canónica de la dependabilidad (Avizienis et al., 2004) al eje operador  $\times$  almacenamiento  $\times$  visibilidad.

*Consistencia*: se expresa según la Ec. (3),

$$C(S) = f(Tx_{\text{confirmadas}}, Tx_{\text{visibles}}), \quad (3)$$

estableciendo como condición fundamental que toda transacción confirmada debe permanecer visible después de un evento de fallo. Este planteamiento se fundamenta en los principios de consistencia en sistemas distribuidos y en el teorema CAP (Gilbert & Lynch, 2002; Bailis & Ghodsi, 2013).

*Rendimiento*: se aproxima según la Ec. (4),

$$P(S) = f(\text{latencia, throughput}), \quad (4)$$

lo que refleja cómo las decisiones de diseño y las características del almacenamiento influyen en la eficiencia del sistema, en línea con estudios comparativos de arquitecturas de bases de datos (Khan et al., 2023).

*Interacción multiplicada*: se representa según la Ec. (5),

$$I(S) = f(O \times K \times M \times D), \quad (5)$$

que captura la dependencia entre operador, plano de control, almacenamiento y base de datos. La inclusión explícita de  $K$  refleja su papel activo —punto único de coordinación durante una partición (Tabla 1)—, no meramente pasivo. Las funciones  $f(\cdot)$  son organizativas: identifican las variables de cada dimensión, no relaciones cerradas ni derivadas analíticamente; su especificación operacional queda como trabajo futuro.

### 3.4. Invariantes del sistema

El modelo se complementa con un conjunto de invariantes que permiten caracterizar un comportamiento aceptable del sistema bajo condiciones normales y de fallo:

- *Invariante de consistencia*: toda transacción confirmada debe permanecer accesible tras un proceso de recuperación, en concordancia con los modelos de consistencia en sistemas distribuidos (Bailis & Ghodsi, 2013).
- *Métrica de disponibilidad*: el tiempo de recuperación (RTO) se reporta como cantidad observable por escenario de fallo. En ausencia de un objetivo de nivel de servicio (SLO) declarado a priori, se enuncia como métrica medida y contrastada entre escenarios —no como umbral duro de conformidad—; un despliegue concreto puede elevarla a invariante atándola a un SLO objetivo (Avizienis et al., 2004).

- *Invariante de durabilidad*: toda operación registrada en el *log* de escritura anticipada (WAL) debe ser recuperable tras un fallo, tal como se establece en estudios sobre mecanismos de recuperación en bases de datos (Stonebraker & Kemnitz, 1991; Han et al., 2024).

Cada invariante se operacionaliza en las Secciones 5 y 6 mediante métricas observables (RPO, latencia de *failover*, pérdida de transacciones).

## 4. Análisis comparativo y discusión de escenarios

### 4.1. Comparación entre operadores de PostgreSQL

Las variaciones en el diseño del plano de control y en el modelo de dependencias condicionan la gestión de fallos. De los operadores *cloud-native* como CNPG cabe esperar mayor coherencia con el ciclo de vida de Kubernetes; de los que dependen de Patroni (Zalando), un efecto en la latencia de detección y promoción; y de los híbridos (Crunchy), un comportamiento intermedio. Estas afirmaciones se derivan del comportamiento documentado —no medido aquí, salvo para CNPG (Tabla 1)— y muestran que la resiliencia  $R(S)$  depende tanto del operador como de su articulación con la coordinación, la replicación y la recuperación (Dobies & Wood, 2020; Kubernetes, 2026b).

### 4.2. Impacto del tipo de almacenamiento CSI

El tipo de almacenamiento introduce variaciones en latencia, durabilidad y dominio de fallo. El local reduce la latencia (favorece  $P(S)$ ) pero expone a fallos de nodo (afecta  $R(S)$ ); el distribuido aporta persistencia y replicación a costa de latencia que puede afectar la visibilidad de las transacciones (Gilbert & Lynch, 2002; Bailis & Ghodsi, 2013); y el de red, menos predecible, introduce variabilidad en  $I(S)$  según la conectividad (Avizienis et al., 2004).

### 4.3. Interacción operador–almacenamiento

Uno de los aspectos más relevantes que surgen de este análisis es la dependencia entre el operador y el tipo de almacenamiento utilizado. Esta interacción resulta especialmente crítica, ya que el comportamiento del sistema no puede explicarse de forma aislada en cada capa. La Tabla 1 sintetiza, a partir del comportamiento documentado de CloudNativePG (CloudNativePG, 2026), Zalando Postgres Operator (Patroni) (Patroni, 2026) y Crunchy Postgres for Kubernetes (Crunchy Data, 2026), la distribución de responsabilidades de detección y recuperación entre capas para los principales tipos de fallo.

La Tabla 1 instancia  $I(S)$ : ante el fallo del pod primario,  $O$  detecta y promueve,  $K$  restablece el enrutamiento,  $M$  reasocia el volumen y  $D$  retoma escrituras; el RTO emerge de la composición temporal de estas acciones, no de una por separado. Ante una partición, el cercado (*fencing*) expone un contraste de diseño: Patroni delega la fuente de verdad en un DCS por consenso (etcd/Raft (Ongaro & Ousterhout, 2014)), mientras que CloudNativePG usa el estado reconciliado del *API server*; ambos evitan el *split-brain* por medios distintos.

### 4.4. Implicaciones para la evaluación del sistema

El análisis pone de manifiesto que la evaluación de sistemas PostgreSQL en Kubernetes no puede abordarse desde una única dimensión: se observa un *trade-off* entre rendimiento, resiliencia y consistencia, condicionado por la combinación de operador y tipo de almacenamiento (Gilbert & Lynch, 2002; Avizienis et al., 2004; Bailis & Ghodsi, 2013). Estos factores se cuantifican empíricamente en la Sección 6 mediante métricas como RTO, RPO, latencia de transacciones y pérdida de datos.

## 5. Metodología experimental

### 5.1. Entorno, aislamiento y alcance

Con el fin de confrontar empíricamente el marco anterior con el comportamiento observa-

do, se ejecutó un experimento sobre un clúster Kubernetes en producción (v1.34.6), con PostgreSQL 16.13 y almacenamiento en red respaldado por el controlador CSI de Huawei 4.10.1 ([csi.huawei.com](https://csi.huawei.com)) sobre *Fibre Channel* (SAN). El almacenamiento se aprovisionó a través de una *StorageClass* con una política de retención de volúmenes (*Retain*) y un enlace diferido al primer consumidor (*WaitForFirstConsumer*), y la red se gestionó mediante Calico 3.31.4. El estudio empírico se realizó sobre CloudNativePG (CNPG) 1.28.0, exponente del enfoque *cloud-native* de la taxonomía de la Sección 3.

El alcance se limitó a un único operador por una restricción del entorno. El clúster era productivo y su equipo de operaciones no permitía instalar operadores nuevos —como Zalando Postgres Operator (Patroni/Spilo) o Crunchy Postgres for Kubernetes— por el riesgo que ello implicaba sobre las cargas en producción; el operador CNPG ya presente era, además, compartido con otros clústeres del entorno. Por tanto, el contraste con operadores basados en Patroni y con enfoques híbridos se aborda de forma analítica en la Tabla 1 —a partir de su comportamiento documentado— y se asigna a la Fase 2, a ejecutar en un entorno no productivo que permita su instalación. Las secciones posteriores remiten a esta justificación sin reiterarla.

La inyección se realizó con Chaos Mesh 2.8.3 en un entorno *air-gapped* (imágenes importadas localmente). El experimento se ejecutó en un espacio de nombres dedicado, con cuotas y control de acceso, de modo que las cargas productivas quedaron fuera del dominio de fallo. El aislamiento descansó en tres controles —Chaos Mesh restringido al espacio de nombres, un doble filtro de selección (espacio de nombres y clúster) en cada manifiesto, y una verificación previa (*dry-run*) de cada selector— y en excluir los pods de la malla (Linkerd) para no sesgar las mediciones de RTO y latencia (el detalle, en el material suplementario).

| Tipo de fallo            | Operador ( <i>O</i> )                                                                                                                                                                                                                                        | Kubernetes ( <i>K</i> )                                                                                                                                                                                           | Almacenamiento CSI ( <i>M</i> )                                                                                                                  |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Fallo del pod primario   | Detecta la pérdida del líder y promueve una réplica (CloudNativePG: gestor de instancias propio; Zalando y Crunchy; Patroni)                                                                                                                                 | Reinicia el pod ( <i>kubelet</i> ) y actualiza <i>Services</i> y <i>Endpoints</i> hacia el nuevo primario                                                                                                         | Reasocia el PVC existente al pod recreado; sin acciones sobre los datos                                                                          |
| Fallo de nodo            | Ordena la promoción de una réplica si el primario residía en el nodo afectado                                                                                                                                                                                | Marca el nodo <i>NotReady</i> ( $\approx 40$ s, <i>node-monitor-grace-period</i> ) y desaloja los pods tras el umbral de tolerancia ( $\approx 5$ min por defecto, <i>tolerationSeconds</i> ) (Kubernetes, 2026a) | Local: datos inaccesibles hasta recuperar el nodo; distribuido: las réplicas permiten reasociar ( <i>detach/attach</i> ) el volumen en otro nodo |
| Fallo de disco / volumen | Recrea la instancia y solicita resincronización desde el primario o desde respaldo (Crunchy: pgBackRest)                                                                                                                                                     | Reporta el PV como degradado y reprograma el pod según la <i>StorageClass</i>                                                                                                                                     | Distribuido (Ceph/Longhorn): reconstruye réplicas del volumen; local: pérdida del volumen, la durabilidad depende del WAL replicado              |
| Partición de red         | Debe evitar el <i>split-brain</i> : cercado ( <i>fencing</i> ) del antiguo primario (Patroni: expiración del <i>lease</i> en un DCS basado en consenso —etcd/Raft (Ongaro & Ousterhout, 2014)—; CloudNativePG: estado reconciliado en el <i>API server</i> ) | El <i>API server</i> actúa como punto único de coordinación; una minoría del plano de control bloquea la reconciliación                                                                                           | En red: E/S bloqueada o degradada; distribuido: la pérdida de quórum puede congelar las escrituras                                               |

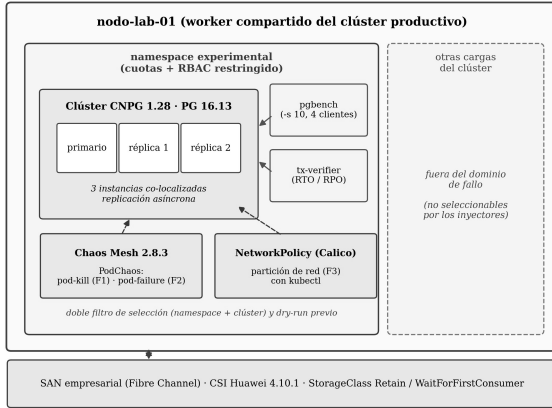
**Tabla 1.** Responsabilidad de detección y recuperación por capa y tipo de fallo.

**Nota:** síntesis analítica del comportamiento documentado por capa ante cada tipo de fallo; no es una medición empírica. *O* = operador, *K* = plano de control de Kubernetes, *M* = almacenamiento CSI. Fuente: documentación de CloudNativePG (CloudNativePG, 2026), Patroni (Patroni, 2026) y Crunchy (Crunchy Data, 2026); parámetros de desalojo de Kubernetes (Kubernetes, 2026a).

El manifiesto del clúster experimental solicitaba antiafinidad entre instancias (*topologyKey*: `kubernetes.io/hostname`), pero el *pool* de nodos elegibles del laboratorio se reducía a nodo-lab-01. En consecuencia, las tres instancias (un primario y dos réplicas) residieron en ese único nodo (Fig. 1). El *failover* observado fue, por tanto, intranodo, y la tolerancia a un fallo de nodo completo no era *testable* por construcción en este entorno. Esta restricción refuerza la lectura del escenario (ii) como cota inferior del RTO ante pérdida de nodo (véanse la Sección 6 y las limitaciones de la Sección 7).

## 5.2. Carga de trabajo y cliente de verificación

El operador desplegó un clúster de tres instancias sometido a una carga sintética continua generada con `pgbench` (escala `-s 10`, 4 clientes y 2 hilos concurrentes), en rondas sucesivas de 60 segundos que toleraban *failovers*. La replicación entre instancias se mantuvo en su modo asíncrono por defecto (PostgreSQL Global Development Group, 2024); el modo síncrono no se varió en este piloto, de modo que las cifras de RPO reportadas corresponden a replicación asíncrona.



**Fig. 1.** Banco de pruebas: clúster experimental colocalizado en nodo-lab-01, con clientes de carga (pgbench) y verificación (tx-verifier), e inyectores de fallo con doble filtro de selección; las cargas productivas quedan fuera del dominio de fallo.

En paralelo, un cliente de verificación (*tx-verifier*) insertó identificadores BIGINT monótonos y anotó cada COMMIT con su marca de tiempo; ante un fallo reintentó el mismo identificador, de modo que un hueco en la tabla de verdad equivale a una escritura reconocida y perdida (*proxy* del RPO). El reintento resuelve el caso frontera del ACK perdido: si el COMMIT sí persistió, la clave duplicada lo revela como «*commit fantasma*» y se contabiliza como tal, no como pérdida, de modo que el RPO no se sobreestima (el detalle, en el material suplementario). La granularidad de medición del RTO —el hueco entre COMMIT consecutivos, con una espera de 0,2 s ante fallo— fue de  $\approx 0,2$  s.

Este diseño operacionalizó los invariantes de la Sección 3: el RPO como transacciones confirmadas no visibles tras el fallo y el RTO como el hueco entre la última escritura reconocida y la primera aceptada tras la recuperación; medido con el reloj único del cliente, el RTO no depende de la sincronización de relojes entre nodos.

### 5.3. Escenarios de fallo y tamaño muestral

Se aplicaron tres escenarios de fallo reproducibles de forma segura en un entorno produc-

tivo, que se corresponden con las métricas F1, F2 y F3 empleadas en los resultados: (i) F1, la eliminación del pod primario, mediante el experimento PodChaos en modo *pod-kill* de Chaos Mesh; (ii) F2, la indisponibilidad sostenida del primario durante 10 minutos, mediante PodChaos en modo *pod-failure*, como aproximación observable de un fallo de nodo desde la perspectiva del operador; y (iii) F3, la partición de red del primario mediante una *NetworkPolicy* de Calico —sin intervención de Chaos Mesh—, aplicada y revertida mediante `kubectl`.

Se planificó un cuarto escenario (F4): sensibilidad del RTO a la latencia de E/S (0, 20, 50 y 100 ms) mediante IOChaos a nivel de FU-SE, que resultó no ejecutable sobre CNPG (Sección 6.2) y se le asigna a la Fase 2.

F1 y F2 se repitieron diez veces cada uno ( $n = 10$ ); F3, diez veces fijas de 60 s más dos inyecciones exploratorias (149 s y 300 s) excluidas del RTO, sin promoción en las doce (0/12). El tamaño muestral respondió a las ventanas de mantenimiento del clúster productivo;  $n = 10$  caracteriza la tendencia central y la dispersión, no efectos marginales, y por la no normalidad esperada se reportaron medianas y rangos intercuartílicos.

Los fallos de nodo —expulsión de pods y reasociación (*detach/attach*) de volúmenes— no se reprodujeron: el clúster productivo no permite drenar nodos y las tres instancias residían en un único nodo elegible (Sección 5.1). Por ello, el escenario (ii) no captura la latencia de reasociación del volumen CSI —dominante en un fallo de nodo genuino— y debe leerse como cota inferior del RTO real; el fallo de nodo se aborda analíticamente (Tabla 1).

## 6. Resultados

Las mediciones sobre CNPG fueron descriptivas (medianas, IQR, intervalos de confianza de mediana) más un único contraste de dos grupos intra-CNPG (F1 frente a F2); por la no normalidad de RTO y RPO se reportaron medianas e

IQR.

### 6.1. Tiempo de recuperación (RTO) y pérdida de datos (RPO)

La Tabla 2 resume los tres escenarios. El RPO fue nulo en todos, pero solo F1 puso a prueba la durabilidad de forma no trivial: al promover una réplica, RPO = 0 implica que ninguna escritura reconocida sobrevivió solo en el primario caído (en F2 y F3 no hubo promoción, así que el RPO nulo es trivial). Que CNPG preserve RPO = 0 en F1 bajo replicación asíncrona es un hallazgo genuino —no una garantía de diseño—, atribuible a un *lag* de replicación  $\approx 0$  bajo la carga ligera empleada (una inferencia de mecanismo, no medida: no se registró el LSN *lag*) y acotado por la colocación intranodo, que evita el enlace de red internodo (se retoma en la Sección 7). Al término del piloto, la tabla de verdad resultó contigua —identificadores 1 a 613 253, sin huecos—: ninguna transacción confirmada se perdió. La variabilidad se concentró, pues, en el RTO.

En F1 (*pod-kill*), CNPG ejecutó *failover* en las diez repeticiones con un RTO mediano de **7,91 s** (IQR  $\approx [7,62, 8,01]$  s; rango 6,61–8,17). F2 (*pod-failure*) no promovió en ninguna: el operador recreó el pod primario en su sitio —mismo nombre e idéntico PVC— con una indisponibilidad mediana de **36,75 s** (IQR  $\approx [36,2, 37,1]$  s; rango 35,5–38,3). F3 tampoco promovió: el pod aislado permaneció *Ready* y la indisponibilidad igualó la duración de la partición (60 s  $\rightarrow$  60,75 s de mediana), con recuperación  $< 1$  s al retirar la *NetworkPolicy*. Los intervalos de confianza de mediana (de distribución libre, cobertura  $\approx 97,9\%$ ) y demás detalles estadísticos se recogen en el material suplementario.

La primera inyección de F2 ( $\approx 80,5$  s, arranque en frío) se excluyó de la mediana, de forma simétrica a las dos exploratorias de F3; la exclusión es conservadora —incluirla *ampliaría* la brecha F1–F2 (material suplementario)—. Dentro del lote de F2 se observó una asociación

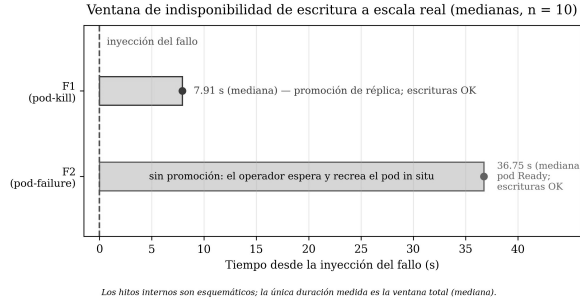
| Esc. | ¿Prom.? | RTO /<br>indisp.     | RPO | Comport.<br>observado |
|------|---------|----------------------|-----|-----------------------|
| F1   | Sí      | 7,91 s               | 0   | Failover rápido       |
| F2   | No      | 36,75 s              | 0   | Recreación in situ    |
| F3   | No      | = duración partición | 0   | CP (consistencia)     |
| F4   | —       | No ejecutable        | —   | Bloqueo (FUSE)        |

**Tabla 2.** RTO y RPO por escenario de fallo (CloudNativePG).

**Nota:** Escenarios: F1 = *pod-kill* (eliminación del primario); F2 = *pod-failure* (indisponibilidad sostenida); F3 = partición de red (*NetworkPolicy* de Calico); F4 = latencia de E/S (IOChaos FUSE). RTO = tiempo hasta restablecer la escritura, medido como la mediana del hueco entre transacciones confirmadas por el cliente de verificación, con una granularidad de instrumento de  $\approx 0,2$  s (véase la Sección 5.2); RPO = transacciones confirmadas no visibles tras el fallo (huecos en la secuencia de la tabla de verdad). «¿Prom.?» indica si el operador promovió una réplica a primario; «CP» denota el comportamiento observado en que el sistema privilegia la consistencia sobre la disponibilidad —el primario aislado no acepta escrituras por ser inalcanzable, sin *split-brain*—, en sentido operacional y no como afirmación formal sobre la posición del sistema en el teorema CAP. Tamaños muestrales: F1 y F2,  $n = 10$ ; F3, 10 repeticiones fijas de 60 s más 2 inyecciones exploratorias, excluidas del cálculo del RTO (promoción en 0/12). El escenario F4 no fue ejecutable (véase la Sección 6.2). El RTO de F2 es una cota inferior del real ante un fallo de nodo genuino (no captura la latencia *detach/attach* del volumen CSI; Sección 7). Fuente: mediciones del piloto.

orden–RTO débil (Spearman  $\rho \approx 0,62$ ), no significativa con  $n = 10$  ( $\rho$  crítico  $\approx 0,65$ ); es un orden de magnitud menor que la separación F1–F2 ( $\approx 29$  s), por lo que no compromete el contraste.

Emergió un resultado contraintuitivo: la recuperación tras *pod-kill* (F1) fue  $\approx 4,65\times$  más rápida que tras *pod-failure* (F2) —36,75 frente a 7,91 s— pese a tratarse del mismo componente (Fig. 2). La separación fue completa: Mann–Whitney  $U = 0$ , correlación rango-biserial 1,00 y diferencia de Hodges–Lehmann de 28,96 s (el



**Fig. 2.** Ventana de indisponibilidad de escritura a escala real: F1 (7,91 s, promoción de réplica) frente a F2 (36,75 s, recreación in situ del pod). Fuente: mediciones del piloto.

$p$  exacto,  $\approx 1,1 \times 10^{-5}$ , es el mínimo alcanzable con  $n = 10$  —test saturado—, por lo que el peso probatorio recae en el tamaño de efecto y el mecanismo, no en el  $p$ ; una prueba de permutación que respeta el orden preserva la separación: material suplementario). El mecanismo: el *pod-kill* elimina el pod y dispara promoción inmediata, mientras que el *pod-failure* induce a esperar la recreación en su sitio. El RTO de F2 es una cota inferior del real ante fallo de nodo (Secciones 5 y 7).

La no promoción en F2 (0/10) y F3 (0/12) admite cotas superiores unilaterales al 95% (Clopper–Pearson) de  $\leq 25,9\%$  y  $\leq 22,1\%$ , pero la conclusión se apoya sobre todo en el mecanismo observado —recreación con la misma identidad y PVC, o pod aislado *Ready*— y no solo en el conteo.

El hallazgo central que articula estos tres comportamientos es que la variable que determina el *failover* en CloudNativePG no es el fallo en sí, sino su visibilidad ante Kubernetes. En F1, la muerte del contenedor deja el pod en *NotReady*; el operador registra el evento «*Current primary isn't healthy, initiating a failover*» y promueve una réplica en  $\approx 7,9$  s. En F3, las sondas del *kubelet* no se bloquean —el *failsafe* de Calico preserva el tráfico de *host*—, de modo que el pod aislado permanece *Ready*, el operador no emite evento alguno y no promueve: elige consistencia sobre disponibilidad (comporta-

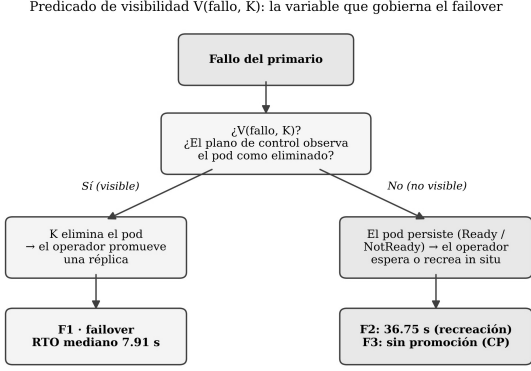
miento CP) sin incurrir en *split-brain*. F2 refina el mecanismo: aunque el pod queda *NotReady*, se recrea con la misma identidad y el mismo PVC, y el operador prefiere esperar esa recreación en su sitio antes que promover. La promoción de F1 se explica, por tanto, por la eliminación del pod —el primario deja de existir— y no por el mero estado *NotReady*.

## 6.2. Sensibilidad a la latencia de E/S

El escenario F4 (sensibilidad del RTO a la latencia de E/S vía IOChaos) resultó no ejecutable sobre CNPG: el inyector de E/S de Chaos Mesh basado en FUSE (*toda*) falló al montar el sistema de archivos con `Read-only file system` porque CNPG endurece sus pods con `readOnlyRootFilesystem: true`, mientras que *toda* requiere una raíz escribible. No fue un error de configuración (el *dry-run* de selectores y el recurso PodIOChaos eran correctos), sino una incompatibilidad entre ese inyector y el endurecimiento del operador —no una propiedad general de Chaos Mesh—. La medición cuantitativa se asigna a la Fase 2 por una vía que no dependa de FUSE (p. ej., `io.max` a nivel de *cgroup*); el detalle completo está en el material suplementario.

## 6.3. Contraste entre el comportamiento anticipado y el observado

El marco anticipaba, por el comportamiento documentado (Tabla 1), que el fallo del pod primario dispara la promoción; F1 confirmó esta expectativa casi trivial. El aporte de mayor peso está en F2: la indisponibilidad sostenida no promovió en ninguna de las diez repeticiones, en contra de la expectativa —derivada de tratarla como aproximación de un fallo de nodo— de que promovería. Un lector familiarizado con *pod-failure* podría anticiparlo; el aporte no es la sorpresa, sino la confirmación empírica y el aislamiento del mecanismo: la promoción no depende del estado *NotReady*, sino de la pérdida de identidad del pod (recreado con el mismo nombre y PVC). F3 exhibió el comportamiento CP —CNPG evita el *split-brain* apoyándose en



**Fig. 3.** Predicado de visibilidad  $V(\text{fallo}, K)$ : la promoción se dispara solo cuando el plano de control observa el pod como eliminado (F1); si el pod persiste (F2, F3), el operador no promueve.

el *API server* (última fila de la Tabla 1)—.

Los tres comportamientos comparten una misma variable de control que el marco actual no captura de forma explícita: la visibilidad del fallo ante Kubernetes (Fig. 3). De ahí que se proponga extender  $S = (O, K, M, D)$  con un predicado de visibilidad  $V(\text{fallo}, K)$ , según la Ec. (6), que registre el estado del pod primario ante el plano de control —eliminado (F1), *NotReady*-recreado (F2) o *Ready* (F3)— como la dimensión que ordena el gradiente de RTO observado:

$$V(\text{fallo}, K) \in \{ \text{eliminado}, \text{NotReady-recreado}, \text{Ready} \}. \quad (6)$$

Es una extensión propuesta motivada por la evidencia, no derivada formalmente. El bloqueo de F4 apunta a una segunda extensión: la *inyectabilidad* de los fallos depende del endurecimiento del operador, algo que el modelo tampoco recoge. No se reclama una validación del marco, sino una caracterización empírica que señale hacia dónde extenderlo.

## 7. Discusión

El comportamiento de PostgreSQL en Kubernetes no se explica por una sola capa: la interacción entre operador, almacenamiento y base de datos introduce efectos emergentes. La con-

fiabilidad *cloud-native* es, por tanto, inherentemente multicapa, y decisiones locales —operador o tipo de almacenamiento— tienen implicaciones globales en RTO, RPO o visibilidad de las transacciones (Gilbert & Lynch, 2002; Avizienis et al., 2004; Bailis & Ghodsi, 2013). La función  $I(S)$  ofrece una base para interpretar estas dependencias, extendiendo enfoques que las analizan por separado.

El alcance es una fase piloto acotada por el entorno productivo (Sección 5). Tres limitaciones, ya detalladas: (i) un único operador (CNPG) y una categoría de almacenamiento —CSI en red—, con la comparación asignada a la Fase 2 (Tabla 1); (ii) el fallo de nodo no se reprodujo —colocalización intranodo—, por lo que el *failover* es intranodo y tanto el RPO nulo de F1 como el RTO de F2 (cota inferior) deben leerse como acotados; y (iii) el análisis de E/S (F4) no fue ejecutable por la incompatibilidad `FUSE/readOnlyRootFilesystem`. Pese a ello, el trabajo conecta distintas líneas y operacionaliza sus invariantes con métricas observables en producción.

La Fase 2 —ya diseñada, con protocolo de ejecución e instrumentación desarrollados— ampliará la validación mediante un diseño factorial  $2 \times 2$  (replicación asíncrona frente a síncrona por quórum  $\times$  topología colocalizada frente a antiafinidad multinodo), incorporando los operadores y categorías de almacenamiento no cubiertos y un escenario de fallo de nodo genuino.

## 8. Conclusiones

Este trabajo abordó los sistemas PostgreSQL en Kubernetes desde una perspectiva multicapa, considerando conjuntamente la interacción entre operadores, almacenamiento basado en CSI y la semántica de la base de datos. Como contribución principal, se presentó una taxonomía y un marco descriptivo —el vocabulario  $S = (O, K, M, D)$  con sus invariantes— junto con un estudio empírico que lo confronta con el comportamiento observado mediante inyección

controlada de fallos sobre un clúster productivo (Secciones 5 y 6), midiendo RTO, RPO y visibilidad de transacciones para CloudNativePG. El hallazgo central es que la variable que gobierna el *failover* no es el fallo en sí, sino su visibilidad ante Kubernetes: la refutación de que la indisponibilidad sostenida del primario promovería una réplica —no lo hizo en ninguna de las diez repeticiones— es el resultado de mayor fuerza probatoria, y motiva extender el marco con un predicado de visibilidad  $V(\text{fallo}, K)$ . La contribución conceptual de mayor alcance es, de hecho, ese predicado de visibilidad; la tupla  $S = (O, K, M, D)$  actúa como andamiaje descriptivo que organiza el análisis, no como un modelo predictivo.

El análisis evidencia que la confiabilidad *stateful* en Kubernetes no depende solo del operador o del almacenamiento, sino del comportamiento emergente de su interacción, con *trade-offs* entre rendimiento, disponibilidad y consistencia que atraviesan capas. El aporte de fondo es doble: acercar la teoría de sistemas distribuidos a la práctica en Kubernetes y entregar una herramienta reutilizable —el modelo  $S = (O, K, M, D)$  con sus invariantes operacionalizados y el predicado  $V(\text{fallo}, K)$ — aplicable a otras configuraciones; su validación se ampliará en la Fase 2 (Sección 7).

## Agradecimientos

El autor utilizó herramientas de inteligencia artificial generativa únicamente para mejorar la redacción, la gramática y el estilo; la concepción, el modelo, la ejecución experimental, el análisis y las conclusiones son obra exclusiva del autor, que revisó y validó todo el contenido (autoría única). La investigación no recibió financiamiento externo y el autor declara no tener conflictos de interés. Los datos limpios de RTO/RPO, el paquete de ejecución (manifiestos, *tx-verifier*, carga pgbench y scripts de análisis) y un material suplementario se depositan públicamente en Zenodo (DOI: 10.5281/zenodo.23004248); los registros crudos del verifica-

dor se facilitan a petición por el acceso restringido del clúster productivo, y el material depositado permite reproducir el diseño en un clúster equivalente.

## Bibliografía

- Alquraan, A., H. Takruri, M. Alfatafta & S. Al-Kiswany. (2018). An analysis of network-partitioning failures in cloud systems. *Proc. 13th USENIX Symp. Operating Systems Design and Implementation (OSDI 18)*. USENIX Association. Carlsbad, CA, USA. pp. 51–68.
- Alvaro, P. & S. Tymon. (2018). Abstracting the geniuses away from failure testing. *Commun. ACM* 61(1): 54–61.
- Avizienis, A., J.-C. Laprie, B. Randell & C. Landwehr. (2004). Basic concepts and taxonomy of dependable and secure computing. *IEEE Trans. Dependable Secure Comput.* 1(1): 11–33.
- Bailis, P. & A. Ghodsi. (2013). Eventual consistency today: limitations, extensions, and beyond. *Commun. ACM* 56(5): 55–63.
- Basiri, A., N. Behnam, R. de Rooij, L. Hochstein, L. Kosowski, J. Reynolds & C. Rosenthal. (2016). Chaos engineering. *IEEE Softw.* 33(3): 35–41.
- Burckhardt, S. (2014). Principles of eventual consistency. *Found. Trends Program. Lang.* 1(1–2): 1–150.
- Burns, B., B. Grant, D. Oppenheimer, E. Brewer & J. Wilkes. (2016). Borg, Omega, and Kubernetes. *ACM Queue* 14(1): 70–93.
- Chen, Z., M. Goudarzi & A. N. Toosi. (2026). An empirical evaluation of Kubernetes resilience in cloud-edge deployments using failure injection techniques. *Proc. 2026 Australasian Computer Science Week (ACSW '26)*. ACM. pp. 22–31.
- CloudNativePG. (2026). CloudNativePG documentation. [En línea]. Disponible: <https://cloudnative-pg.io/docs/> [Consultado: 7 de julio de 2026].
- Crunchy Data. (2026). PGO: the Postgres Operator from Crunchy Data — documentation. [En línea]. Disponible: <https://access.crunchydata.com/documentation/postgres-operator/latest/> [Consultado: 7 de julio de 2026].
- Dobies, J. & J. Wood. (2020). *Kubernetes Operators: automating the container orchestration platform*. O'Reilly Media. Sebastopol, CA, USA.
- Drees, F. (2026). Chaos testing the CloudNativePG project. *CloudNativePG Blog*. [En línea]. Disponible: <https://cloudnative-pg.io/blog/lfx->

- chaos-testing-yash-agarwal/ [Consultado: 8 de julio de 2026].
- Engelbert, C. (2024). How to choose your Kubernetes Postgres operator? *simplyblock Blog*. [En línea]. Disponible: <https://simplyblock.io/blog/choosing-a-kubernetes-postgres-operator/> [Consultado: 7 de julio de 2026].
- Gilbert, S. & N. Lynch. (2002). Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News* 33(2): 51–59.
- Han, F., H. Liu, B. Chen, D. Jia, J. Zhou, X. Teng, C. Yang, H. Xi, W. Tian, S. Tao, S. Wang, Q. Xu & Z. Yang. (2024). PALF: replicated write-ahead logging for distributed databases. *Proc. VLDB Endow.* 17(12): 3745–3758.
- Khan, W., T. Kumar, C. Zhang, K. Raj, A. M. Roy & B. Luo. (2023). SQL and NoSQL database software architecture performance analysis and assessments — a systematic literature review. *Big Data Cogn. Comput.* 7(2): art. 97.
- Kingsbury, K. & P. Alvaro. (2021). Elle: inferring isolation anomalies from experimental observations. *Proc. VLDB Endow.* 14(3): 268–280.
- Kubernetes. (2024). Container storage interface (CSI). *Kubernetes Documentation*. [En línea]. Disponible: <https://kubernetes.io/docs/concepts/storage/volumes/> y la especificación versionada en <https://github.com/container-storage-interface/spec> [Consultado: 7 de julio de 2026].
- Kubernetes. (2026a). Nodes: node controller, condiciones del nodo y desalojo basado en taints. *Kubernetes Documentation*. [En línea]. Disponible: <https://kubernetes.io/docs/concepts/architecture/nodes/> [Consultado: 7 de julio de 2026].
- Kubernetes. (2026b). Operator pattern. *Kubernetes Documentation*. [En línea]. Disponible: <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/> [Consultado: 8 de julio de 2026].
- Mega, C. (2023). Orchestrating information governance workloads as stateful services using Kubernetes operator framework. *Service-Oriented Computing (SummerSOC 2023), Commun. Comput. Inf. Sci.*, vol. 1847. Springer. Cham, Switzerland. pp. 125–143.
- Ongaro, D. & J. Ousterhout. (2014). In search of an understandable consensus algorithm (Raft). *Proc. USENIX Annual Technical Conf. (USENIX ATC)*. USENIX Association. pp. 305–319.
- Patroni. (2026). Patroni: a template for PostgreSQL HA with ZooKeeper, etcd or Consul — documentation. [En línea]. Disponible: <https://patroni.readthedocs.io/> [Consultado: 7 de julio de 2026].
- Portworx. (2024). Kubernetes operator for PostgreSQL: how to choose and set up one. *Portworx Blog*. [En línea]. Disponible: <https://portworx.com/blog/choosing-a-kubernetes-operator-for-postgresql/> [Consultado: 7 de julio de 2026].
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 documentation: streaming replication y synchronous\_commit. [En línea]. Disponible: <https://www.postgresql.org/docs/16/> [Consultado: 7 de julio de 2026].
- Schwarzkopf, M., A. Konwinski, M. Abd-El-Malek & J. Wilkes. (2013). Omega: flexible, scalable schedulers for large compute clusters. *Proc. EuroSys. ACM*. pp. 351–364.
- Stonebraker, M. & G. Kemnitz. (1991). The POSTGRES next generation database management system. *Commun. ACM* 34(10): 78–92.
- Taft, R., I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieser, K. Niemi, A. Woods, A. Birzin, R. Poss, P. Bardea, A. Ranade, B. Darnell, B. Grunier, J. Jaffray, L. Zhang & P. Mattis. (2020). CockroachDB: the resilient geo-distributed SQL database. *Proc. ACM SIGMOD Int. Conf. Management of Data. ACM*. pp. 1493–1509.
- Van Renesse, R. & F. B. Schneider. (2004). Chain replication for supporting high throughput and availability. *Proc. OSDI. USENIX Association*. pp. 91–104.